

DBMS — Write-Read Conflict / Dirty Read ★

Detailed Analysis, Context Switches, Failures & Real-World Examples

1. WHAT IS A WRITE-READ CONFLICT?

A **Write-Read (W-R) Conflict** occurs when one transaction writes a modified value to a data item, and a second transaction reads that value before the first transaction has safely committed.

Core Definition

If the writing transaction later encounters an error and rolls back, the reading transaction has processed uncommitted/temporary data that is no longer valid. This anomaly is universally known as a **Dirty Read** (or *Uncommitted Read*).

Common Terminology Aliases:

- Write-Read Conflict (W-R Conflict)
- Dirty Read Problem
- Uncommitted Read Anomaly
- Read-After-Write (RAW) Dependency

2. WHY DOES THIS PROBLEM OCCUR?

This problem is inherent to **parallel / concurrent schedules** where operations of multiple transactions are interleaved across CPU context switches:

Serial Schedule

SAFE

T1 Complete → T2 Start
Because T1 completely commits or rolls back before T2 executes, T2 never sees uncommitted intermediate writes.

Parallel Schedule

CONFLICT RISK

T1 [Write] → T2 [Read] → T1 [Fail]
Context switches allow T2 to read T1's dirty data before T1 finalizes its status.

3. STEP-BY-STEP EXECUTION TRACE 🔥

Consider two transactions T1 and T2 operating on a shared database item A (Initial Value: $A = 70$).

Init Database state: $A = 70$

T1 READ(A) → Reads 70

T1 $A = A - 50 \rightarrow (70 - 50 = 20)$

T1 WRITE(A) → Updates buffer $A = 20$ [Uncommitted!]

--- **CONTEXT SWITCH TO T2**

T2 READ(A) → Reads 20 [Dirty Data!]

T2 $A = A \times 2 \rightarrow (20 \times 2 = 40)$

T2 WRITE(A) → Updates $A = 40$

T2 COMMIT → T2 completes successfully and saves $A = 40$

--- **CONTEXT SWITCH BACK TO T1**

T1 FAILS! (Power failure / Hardware fault / System error)

CRASH

T1 ROLLBACK → Reverts T1's changes: $A = 20$ is undone back to $A = 70$

ABORT

4. WHERE IS THE ERROR?

- T2 performed its multiplication ($20 \times 2 = 40$) using the dirty value 20.
- Because T1 crashed, its write was undone, restoring $A = 70$.

5. REAL-LIFE EXAMPLE: RAILWAY RESERVATION SYSTEM (IRCTC) 🚂

Dirty reads occur in real-world booking applications when uncommitted temporary holds are observed by other concurrent users:

Scenario: Seat Availability Counter

Initial State: Available Seats = 2

1. **User 1 (T1)** selects both seats during checkout. The system temporarily updates Available Seats = 0.
2. **User 2 (T2)** checks availability concurrently. T2 reads Available Seats = 0 and assumes the train is fully booked.
3. **User 1 (T1)** encounters a payment gateway failure! The system aborts T1 and rolls back Available Seats = 2.
4. **Anomaly:** User 2 was falsely turned away based on User 1's temporary, uncommitted state!

6. CONFLICT VS. SAFE SCHEDULE COMPARISON

A Dirty Read is avoided when the reading transaction is forced to wait until the writing transaction either commits or rolls back:

❌ Unsafe (Dirty Read Occurs)	✅ Safe (No Dirty Read)
T1: WRITE(A) T2: READ(A) [Reads uncommitted write] T1: FAIL & ROLLBACK Result: T2 read invalid data!	T1: WRITE(A) T1: COMMIT [Successfully committed] T2: READ(A) [Reads committed write] Result: T2 read valid, permanent data.

7. CONNECTION WITH ACID PROPERTIES ★

Atomicity Link

Atomicity demands that either *all* operations of a transaction execute or *none* do. When T1 fails, atomicity forces a rollback. Dirty reads violate this isolation because T2 observed T1's partial, discarded state.

Isolation Link

Dirty reads represent a direct failure of the **Isolation** property. To prevent dirty reads, systems use locking (e.g., Strict 2PL) or higher isolation levels (Read Committed).

🎯 EXAM REVISION & MASTER SUMMARY

Exam-Oriented Definition:

A **Dirty Read (Write-Read Conflict)** occurs when a transaction reads data modified by another transaction before that transaction commits. If the writing transaction later fails or rolls back, the value read by the second transaction becomes invalid, compromising database integrity.

🔥 **One-Line Master Rule:** T1 writes → T2 reads uncommitted data → T1 rolls back → T2 read dirty data!